Efficient Vector-Multiplicative Privacy-Preserving Retrieval-Augmented Generation for Large Language Models

Jinhao Zhou, *Graduate Student Member, IEEE*, and Jun Wu, *Senior Member, IEEE*

***Abstract*—Retrieval-Augmented Generation (RAG) grounds large language models (LLMs) in external knowledge, yet it is faces a fundamental trade-off between knowledge confidentiality and retrieval efficiency. Existing approaches fail to reconcile this trade-off: i) Cryptography-based solutions (e.g., homomorphic encryption and secure multi-party computation) provide strong privacy guarantees but incur prohibitive computation and communication overheads. ii) Lightweight perturbation-based methods (e.g., differential privacy) offer higher efficiency at the cost of degraded retrieval accuracy or weakened security guarantees. In this paper, we propose CipheRAG, an efficient vector-multiplicative privacy-preserving RAG framework that achieves a principled balance between robust privacy and high-performance retrieval. Technically, we first propose an efficient searchable inner product functional encryption (IPFE) mechanism enhanced with asymmetric locality-sensitive hashing (ALSH), enabling the retrieval of sensitive knowledge while effectively preserving data confidentiality. Secondly, we propose a decryption-enabled attention mechanism that uses linear weights of the attention layer to decrypt knowledge. This mechanism seamlessly integrates decrypted knowledge into the LLM's generation process, achieving efficiency and accuracy. Extensive experiments demonstrate that CipheRAG achieves up to 35x faster generation and 15x faster QKV computation compared to FHE- and OT-based baselines. By avoiding linear retrieval and full-parameter encryption, CipheRAG enables privacy-preserving RAG with bounded computational and communication overheads, making it well-suited for deployment in privacy-sensitive environments.**

***Index Terms*—Large language model (LLM), inner product encryption, retrieval-augmented generation (RAG), locality-sensitive hashing.**

I. Introduction

THE retrieval-augmented generation (RAG) [1] technique has emerged as a promising paradigm to enhance the capabilities of large language models (LLMs) services [2], [3], [4], [5]. By incorporating externally sourced information, referred to as "knowledge", RAG allows LLMs to fetch contextually relevant data from various knowledge repositories. As illustrated in Fig. 1, a typical RAG framework maintains a central knowledge database compiled from multiple providers. When a user submits a query, the LLM first consults this database to retrieve the most pertinent information, which it then integrates into its reasoning process to produce a more informed and accurate response. This ability to dynamically tap into external knowledge underlies the strength of RAG, making it a promising approach for building LLM applications that demand up-to-date, context-aware information [6].

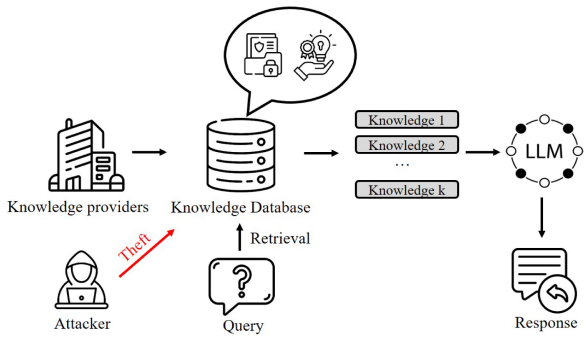

Fig. 1. The workflow of a RAG system. A potential attacker threatens the privacy of the knowledge database.

Despite their effectiveness, RAG systems face persistent privacy risks because the retrieved knowledge often contains sensitive information. Such sensitive information can include confidential documents, domain-specific corpora, or intellectual property [7], which is vulnerable to attacks including embedding inversion [8] and knowledge poisoning [9]. Moreover, any leakage during retrieval can propagate into the downstream LLM, compromising model security and enabling inference attacks that threaten both the integrity of the model and the privacy of its users [10], [11], [12].

The consequences of such breaches are severe. For instance, when an authorized medical professional queries an RAG system to review the patient's medical records or specialist recommendations, the underlying dataset encompasses information that must remain confidential. Unauthorized access to such information can result in severe consequences, encompassing substantial legal penalties, erosion of patient trust, and considerable reputational harm. Therefore, it is essential to design a retrieval primitive that inherently protects both vector representations and metadata, for example by operating entirely over encrypted or enclave-protected data, to eliminate the principal leakage

Received 24 December 2024; revised 11 January 2026; accepted 15 February 2026. Date of publication 3 March 2026; date of current version 12 May 2026. This work was supported in part by JSPS KAKENHI under Grant 23K11072 and in part by the China Scholarship Council Program. *(Corresponding author: Jun Wu.)*

The authors are with the Graduate School of Information, Production and Systems, Waseda University, Fukuoka 169-8050, Japan (e-mail: jinhao@suou.waseda.jp; junwu@aoni.waseda.jp).

Digital Object Identifier 10.1109/TDSC.2026.3669543

channel without compromising the accuracy of retrieved knowledge, which is crucial for the generation quality of LLM services. A direct way to protect privacy is by adding perturbations to the data [13]; however, this approach often incurs a significant accuracy drop.

Encryption-based methods, which offer robust data confidentiality by enabling computations directly on encrypted data, have been proposed to ensure privacy protection while maintaining accuracy. They provide a promising solution for securely performing retrieval and generation tasks within RAG systems without exposing additional information. Homomorphic encryption (HE) [14] and secure multi-party computation (SMPC) [15] have been widely adopted to secure sensitive information during computation. For instance, CipherGPT [16], EncryptedLLM [17], and EncChain [18] integrate secure matrix multiplication or fine-grained access control with FHE to handle data within RAG frameworks securely. More recently, distributed frameworks like FRAG [19] and BumbleBee [20] support secure approximate k-nearest neighbor searches over encrypted vector databases, while PIR-RAG [21] leverages private information retrieval to strictly hide access patterns.

However, the substantial computational overhead of these encryption-based methods limits the performance and practicality of RAG systems. This challenge arises because LLMs rely heavily on computationally intensive operations, such as inner products and matrix multiplications, which become significantly more complex and time-consuming when performed on encrypted data [22], [23]. Consequently, a critical gap remains for methods capable of ensuring strong privacy protection without sacrificing retrieval efficiency or accuracy. Addressing this gap requires novel encryption mechanisms and computational strategies that can efficiently operate on encrypted data, preserving the quality and responsiveness essential for real-world LLM applications.

To address these challenges, we present CipheRAG, an efficient vector-multiplicative privacy-preserving RAG framework for LLMs. The key aspect is the secure computation of encrypted data through efficient inner product operations, ensuring fast and integral retrieval and generation. First, to ensure privacy protection while enabling efficient retrieval, we present an asymmetric locality-sensitive hashing (ALSH) [24] enhanced inner product functional encryption (IPFE) [25] storage mechanism. By hashing and encrypting knowledge, we keep sensitive information confidential and maintain fast data retrieval. Second, to preserve the integrity of encrypted knowledge and maintain generation performance, we propose a decryption-enabled attention layer that leverages linear "query", "key", and "value" weights [26] to transform and decrypt the ciphertext. This partial encryption strategy significantly reduces the computation overhead over encrypted data.

Compared to existing approaches, CipheRAG is preferable because the use of ALSH and IPFE ensures that retrieval remains as efficient and accurate as traditional unencrypted systems, overcoming the major computational limitations of prior encryption-based techniques. This makes CipheRAG particularly well-suited for practical deployment in privacy-sensitive LLM services, where both strong confidentiality and responsiveness are required. Our objectives are twofold: first, to guarantee that all knowledge and its associated embeddings remain encrypted before storage, thereby preventing any unauthorized access to plaintext data; second, to enable secure, end-to-end computation over ciphertext, so that LLMs can generate responses without ever exposing the underlying knowledge.

The main contributions are summarized as follows:

- We propose the CipheRAG framework, an efficient vector-multiplicative privacy-preserving RAG framework, which enables both efficient and integral retrieval and generation of encrypted knowledge.
- We propose a novel partial encryption strategy for LLMs. In contrast to existing methods that rely on full encryption and hinder computational efficiency, our strategy encrypts input data by IPFE while incorporating a decryption-enabled attention mechanism within the early layers of the LLM. This innovative strategy ensures robust data privacy while maintaining high-generation efficiency.
- We conduct a rigorous theoretical analysis to establish the security properties of CipheRAG. Additionally, we perform experiments to demonstrate its efficient retrieval capabilities and its effectiveness in enhancing the performance of LLMs.

The rest of this paper is organized as follows. Section II reviews related work. Preliminaries are provided in Section III. The threat model and design goals are introduced in Section IV, and the proposed framework is detailed in Section V. Theoretical analyses are presented in Section VI. Experimental results are reported in Section VII. Further discussion is presented in Section VIII. Finally, Section IX concludes the paper and discusses the limitations.

II. Related Work

In this section, we introduce preliminaries about RAG, the privacy risks, and the privacy-preserving RAG system.

A. Retrieval-Augmented Generation

RAG has emerged as a pivotal technique for enhancing LLM performance by integrating external knowledge sources into the generation process [27]. The foundational work by Lewis et al. (2020) [1] introduced RAG as a method that combines retrieval mechanisms with generative models to improve responses on knowledge-intensive tasks. By leveraging a retrieval component, RAG enables models to access and incorporate relevant information dynamically, extending their capabilities beyond the static knowledge captured during training. Subsequent research has built upon this framework to address various challenges and expand its applications. For example, Guu et al. (2020) [7] proposed a retrieval-based pre-training approach that enhances open-domain question-answering performance by allowing the model to access a large corpus of text during inference. Similarly, Izacard et al. (2021) [28] developed a retrieval-augmented reader that processes retrieved documents jointly, improving the model's ability to generate informed and contextually appropriate answers.

Despite these advancements, existing RAG systems face significant challenges in securely handling sensitive data.

Integrating external data sources raises concerns about privacy and data confidentiality, particularly when dealing with personal or proprietary information [29]. Compared to existing works, our research offers a general solution by integrating efficient privacy-preserving retrieval and generation.

B. Privacy Risks of RAG System

Although LLMs demonstrate strong performance, they can unintentionally memorize and reveal fragments of training data, underscoring the need to protect sensitive information against membership inference attacks [30], [31], [32]. Incorporating RAG frameworks introduces additional attack surfaces, as detailed in recent surveys [33]. Malicious queries against proprietary knowledge bases can force the model to disclose restricted content or leverage indirect prompt injections to manipulate outputs [10], [23].

Crucially, recent studies highlight the specific vulnerability of vector embeddings. While often perceived as opaque, embeddings can be subjected to inversion attacks to reconstruct the original raw text with high fidelity [34]. More alarmingly, Huang et al. (2024) [8] recently demonstrated that such embedding inversion attacks are transferable, posing severe privacy risks even without direct query access to the target model. Furthermore, without integrity checks, RAG systems are susceptible to data poisoning, where adversaries inject malicious documents to corrupt retrieval results [9]. Consequently, an ideal defense mechanism must simultaneously address three critical threats: unauthorized leakage, vector inversion, and tampering with the knowledge base.

C. Privacy-Preserving RAG System

Integrating RAG within LLMs necessitates robust security measures to safeguard sensitive data. Traditional RAG systems often expose vulnerabilities that can lead to unauthorized access or leakage of confidential information [29], [35], [36]. To address these concerns, encryption-based methods have emerged as a key direction. Zhu et al. (2024) [37] introduce a semantic search mechanism designed to run on encrypted data, enabling accurate and privacy-preserving retrieval. Expanding on encrypted interaction, Zyskind et al. (2023) [38] employ SMPC to distribute queries securely across a privately constructed database. Similarly, Zhao [19] and Lu et al. (2025) [20] advocate for encrypted approximate k-nearest neighbor searches across dispersed data without compromising confidentiality.

Recent advancements have further pushed the boundaries of FHE for LLMs. Hou et al. (2023) [16] present CipherGPT for secure matrix multiplication. Building on this, EncryptedLLM [17] and Ruoyan et al. (2025) [39] leverage GPU acceleration to mitigate the heavy computational overhead of FHE-based inference. Furthermore, Bae et al. (2025) [40] propose a privacy-preserving interaction framework combining homomorphically encrypted vector databases with Socratic chain-of-thought reasoning. In addition, Fu et al. (2024) [18] propose EncChain to ensure data remains secure throughout model outputs, while PIR-RAG [21] strengthens security by leveraging private information retrieval to hide access patterns. Another type of approach relies on randomization-based methods or data sanitization. DP [13], [41] is widely used to anonymize sensitive information. Yu et al. (2024) [42] and Koga et al. (2025) [43] propose textual DP mechanisms for RAG systems, masking sensitive content while attempting to maintain reasoning capabilities. Complementary to these algorithmic defenses, Zeng et al. (2025) [44] explore a data-centric approach, mitigating privacy issues by substituting sensitive retrieval corpora with high-utility synthetic data. However, as noted by Majmudar et al. (2022) [45], perturbation and sanitization methods can struggle to sustain the quality and coherence of generated text due to the introduced noise or information loss.

TABLE I
QUALITATIVE COMPARISON OF PRIVACY-PRESERVING RAG SCHEMES

Scheme	Year	Comp. Complexity (Encrypted Ops)	Comm. Cost	Retrieval Complexity	Privacy Guarantee
CipherGPT [16]	2023	High	High (Multi-round)	Linear $O(N)$	Strong
BumbleBee [20]	2025	High	High	Linear $O(N)$	Strong
EncryptedLLM [17]	2025	High	High	Linear $O(N)$	Strong
PIR-RAG [21]	2025	Moderate (linear scan)	High	Linear $O(N)$	Strong
Textual DP [43]	2024	N/A	Low	Sub-linear (ANN)	Moderate (Utility Loss)
CipheRAG (Ours)	**2025**	Low (partial encryption, one-time)	**Low (One-pass)**	**Sub-linear** $O(N^{\rho})$	**Strong**

Note: N is the database size.

Table I provides a qualitative comparison between CipheRAG and recent state-of-the-art secure RAG frameworks. While prior works have made progress in protecting retrieval and inference, their practical deployment is predominantly hindered by the high cost of encrypted inference, where full-parameter encryption incurs prohibitive matrix-matrix operations and multi-round interaction overhead. In contrast, CipheRAG fundamentally rethinks the inference-time encryption paradigm by adopting a partial encryption strategy that confines cryptographic computation to lightweight vector-level projections, while allowing the remaining attention and generation processes to be executed in plaintext. This design significantly reduces inference overhead and communication latency, and is complemented by an efficient sub-linear retrieval mechanism based on Asymmetric LSH. Together, these components enable CipheRAG to achieve scalable and efficient secure inference without sacrificing rigorous privacy guarantees.

III. PRELIMINARIES

In this section, we introduce the notations and some related cryptographic primitives.

A. Notations

The notation table is illustrated in Table II.

B. Related Cryptographic Primitive

1) Decisional Composite Residuosity (DCR) Assumption: The Ada-IPFE scheme is instantiated over the Paillier cryptosystem, leveraging the decisional composite residuosity (DCR) assumption. All cryptographic operations, including key generation, encryption, and decryption, are conducted within the

TABLE II
SUMMARY OF NOTATIONS

Symbol	Description
λ	Security parameter
N	RSA modulus for Ada-IPFE, product of two safe primes
g, g'	Public generators in Ada-IPFE
η_i	Public parameter for the i-th coordinate, $g^{s_i} \bmod N^2$
$\mathbf{s}$	Master secret key vector in Ada-IPFE
$\mathbf{x}$	Plaintext (knowledge) vector in $\mathbb{Z}_N^n$
$\mathbf{y}$	Query or attribute vector in $\mathbb{Z}_N^n$
mpk	Master public key
msk	Master secret key
ct	Ciphertext of $\mathbf{x}$ in Ada-IPFE
$\mathrm{sk}_{\mathbf{y}}$	Functional secret key for query vector $\mathbf{y}$
r, a, b, c	Random values used in Ada-IPFE encryption
$h_{\mathbf{a},b}(\mathbf{v})$	L2-norm or asymmetric locality-sensitive hash function
$\mathbf{a}$	Random vector drawn from $\mathcal{N}(0,1)^n$ in LSH
b	Random shift parameter for LSH
r	Bucket width parameter in LSH
$F_r(d)$	Collision probability function for LSH
$\Phi(v)$	CDF of the standard normal distribution
$\mathbf{v}, \mathbf{q}$	Vectors in $\mathbb{R}^n$
$\|\mathbf{v}-\mathbf{q}\|_2$	Euclidean distance between vectors $\mathbf{v}$ and $\mathbf{q}$
$\Pr(\cdot)$	Probability of an event

Paillier modulus setting, which is formally defined below. Unless otherwise stated, all probabilistic polynomial-time (PPT) adversaries are considered. Let $N = pq$ be an RSA modulus, where p and q are large primes. The DCR assumption states that, given a random element $z \in \mathbb{Z}_{N^2}^*$, no probabilistic polynomial-time adversary can distinguish whether z is sampled uniformly at random from $\mathbb{Z}_{N^2}^*$, or whether z is of the form $z_0^N \bmod N^2$ for some uniformly random $z_0 \in \mathbb{Z}_N^*$. Formally, for any PPT algorithm $\mathcal{A}$, its advantage in distinguishing the following two distributions is negligible in the security parameter λ:

$$D_1 = \{z \mid z \leftarrow \mathbb{Z}_{N^2}^*\},$$
$$D_2 = \left\{z \mid z = z_0^N \bmod N^2, z_0 \leftarrow \mathbb{Z}_N^*\right\}. \quad (1)$$

That is, the advantage

$$\mathrm{Adv}_{\mathcal{A}}^{\mathrm{DCR}}(\lambda) = \left|\Pr[\mathcal{A}(z \leftarrow D_1) = 1] - \Pr[\mathcal{A}(z \leftarrow D_2) = 1]\right|, \quad (2)$$

is negligible.

2) Adapted Inner Product Functional Encryption (Ada-IPFE): In our work, we adopt an improved inner product functional encryption scheme, denoted as Ada-IPFE [25], which is instantiated over the DCR setting. The scheme enhances resistance to key recovery and linearity attacks, while retaining efficient vector encryption and decryption. Assume that we intend to reveal $\langle \mathbf{z}, \mathbf{y} \rangle$ over $\mathbb{Z}_N$.

Setup phase $\mathtt{Setup}(1^\lambda, \mathrm{Z}, \mathrm{Y})$: Let λ be the security parameter and n the dimension. First, the system generates an RSA modulus $N = \tilde{p} \cdot \tilde{q}$, where $\tilde{p} = 2p' + 1$, $\tilde{q} = 2q' + 1$ are safe primes with p', q' of length at least $2^{l(\lambda)}$ for polynomial l. Second, sample a generator $g' \leftarrow \mathbb{Z}_{N^2}^*$ and define $g = g'^{2N} \bmod N^2$. Third, sample a master secret vector $\mathbf{s} \in \mathbb{Z}^n$ (typically from a discrete Gaussian distribution). Last, compute public elements $\boldsymbol{\eta} = (\eta_1, \ldots, \eta_n)$ with $\eta_i = g^{s_i} \bmod N^2$ for $i \in [n]$.

The public parameters mpk and the master secret key msk are:

$$\mathrm{mpk} = (g, \{\eta_i\}_{i=1}^n, N), \quad \mathrm{msk} = (\mathrm{s}, \mathrm{Y}), \quad (3)$$

where $\mathbf{Y}$ is the upper bound of plaintext.

Key Generation $\mathtt{KeyGen}(y, \mathrm{msk})$: Given a query vector $\mathbf{y} \in \mathbb{Z}^n$, the authority samples independent random values $\alpha, \beta \leftarrow \mathbb{Z}_N$. The functional key is computed as:

$$\mathrm{sk}_{\mathbf{y}} = \langle \mathbf{s}, \mathbf{y} \rangle + \alpha + \beta,$$
$$\mathrm{pk}_1 = g^\alpha \bmod N^2, \mathrm{pk}_2 = g^\beta \bmod N^2. \quad (4)$$

The secret key is $\mathrm{sk}_{\mathbf{y}} = (\beta, \langle \mathbf{s}, \mathbf{y} \rangle + \alpha + \beta)$, and the associated public key is $\mathrm{pk}_{\mathbf{y}} = (\mathrm{pk}_1, \mathrm{pk}_2)$.

Encryption $\mathtt{Enc}(z, \mathrm{mpk}, \mathrm{pk}_y)$: To encrypt $\mathbf{x} \in \mathbb{Z}^n$, choose random values $r \in_R \{0, 1, \ldots, \lfloor N/4 \rfloor\}, a \in_R \mathbb{Z}_N^*, b, c \in_R \mathbb{Z}_N$.

$$\sigma = \mathrm{pk}_2^c \bmod N,$$
$$\mathrm{ct}_1 = g^r \bmod N^2, \mathrm{ct}_2 = g^b \bmod N^2,$$
$$\mathrm{ct}_3 = (c \cdot a - b) \bmod N, \mathrm{ct}_4 = \mathrm{pk}_1^r \bmod N^2,$$
$$\mathrm{ct}_5 = \left\{\mathrm{ct}_i = h_i^r (1 + N\sigma x_i) \bmod N^2\right\}_{i \in [1,n]}. \quad (5)$$

The ciphertext is $ct = (\mathrm{ct}_0, \mathrm{ct}_1, \mathrm{ct}_2, \mathrm{ct}_3, \mathrm{ct}_4, \mathrm{ct}_5)$, where $\mathrm{ct}_0 = a$.

Decryption: $\mathtt{Dec}(\mathrm{sk}_y, \mathrm{ct}_z)$: Given $\mathrm{sk}_{\mathbf{y}} = (\beta, \mathrm{sk})$ and $ct = (a, \mathrm{ct}_1, \mathrm{ct}_2, \mathrm{ct}_3, \mathrm{ct}_4, \mathrm{ct}_5)$, define:

$$\tau_1 = \left(\left(\mathrm{ct}_2 \cdot g^{\mathrm{ct}_3}\right)^{\beta \cdot \mathrm{ct}_0^{-1} \bmod N} \bmod N^2\right) \bmod N$$
$$\tau_2 = \frac{\left(\mathrm{ct}_1^{(\beta - sk) \bmod N} \mathrm{ct}_4 \prod_{i=1}^n ct_i^{y_i} \tau_1^{-1} - 1\right) \bmod N^2}{N} \quad (6)$$

The decrypted result τ_2 yields the inner product $\langle z, y \rangle$. The presence of independent blinding terms $(\alpha, \beta, a, b, c, r)$ in key and encryption ensures that even with multiple key queries or ciphertexts, an adversary cannot recover the underlying vectors or master secret. The semantic security follows from the DCR assumption, and linearity attacks are mitigated by the randomness added at each protocol step.

3) L2-Norm Locality-Sensitive Hashing (LSH) [46]: The L2-norm-based LSH is a technique for approximating nearest neighbor search in high-dimensional spaces. It is particularly useful for tasks such as similarity search and clustering. The hash functions are designed so that similar vectors are more likely to collide (i.e., hash to the same value).

The hash function for L2-norm LSH is defined as:

$$h_{\mathbf{a},b}(\mathbf{v}) = \left\lfloor \frac{\mathbf{a}^\top \mathbf{v} + b}{r} \right\rfloor, \quad (7)$$

where $\mathbf{a} \leftarrow \mathcal{N}(0,1)^n$ is a random vector with each element sampled independently from the standard normal distribution. $b \leftarrow U[0, r)$ is a scalar sampled uniformly at random from the interval $[0, r)$. $r > 0$ is the bucket width parameter.

The collision probability between two vectors $\mathbf{v}, \mathbf{q} \in \mathbb{R}^n$ under this hash function is given by:

$$\Pr\left(h_{\mathbf{a},b}(\mathbf{v}) = h_{\mathbf{a},b}(\mathbf{q})\right) = F_r\left(\|\mathbf{v}-\mathbf{q}\|_2\right), \tag{8}$$

where,

$$F_r(d) = 1 - 2\Phi\left(-\frac{r}{d}\right) - \frac{2}{\sqrt{2\pi}}\left(\frac{1-e^{-\frac{(r/d)^2}{2}}}{r/d}\right), \tag{9}$$

and $\Phi(v) = \int_{-\infty}^{v} \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}} dx$ is the cumulative distribution function (CDF) of the standard normal distribution. The distance $d = \|\mathbf{v}-\mathbf{q}\|_2$ is the Euclidean distance between $\mathbf{v}$ and $\mathbf{q}$. The function $F_r(d)$ is monotonically decreasing in d, indicating that closer vectors have a higher probability of collision. This hashing mechanism transforms the MIPS problem into a distance-based problem.

4) Secure Transformation: In CipheRAG, to enable cryptographic encryption of the data, we map the embedding x into the finite field $\mathbb{Z}_p$, following the method outlined in [47]. Since x may be a decimal number with up to l_D decimal places, we convert it into an integer by multiplying it by 10^{l_D}, yielding $x' = 10^{l_D} \times x$. Similarly, if another decimal number y is involved in the computation, we convert it into an integer as $y' = 10^{l_D} \times y$. We then compute the product $z = x' \times y'$, which corresponds to:

$$z = (10^{l_D}x) \times (10^{l_D}y) = 10^{2l_D}(xy). \tag{10}$$

To obtain an integer representation of xy with l_D decimal places, we need to scale z down by 10^{l_D}. However, since division may not be feasible within the integer domain $\mathbb{Z}_p$, we perform truncation by decomposing z as:

$$z = z_1 \cdot 10^{l_D} + z_2, \tag{11}$$

where $0 \le z_2 < 10^{l_D}$. Here, z_1 represents the integer part of z divided by 10^{l_D}, effectively truncating the last l_D digits of z. The final truncated result z_1 approximates xy scaled by 10^{l_D}, maintaining the desired precision for subsequent computations.

IV. Threat Model and Design Goal

In this section, we present the threat model and design goal.

A. Threat Model

1) Adversary Model: The primary threat to the CipheRAG system comes from malicious attackers seeking unauthorized access to the RAG database. These adversaries can potentially exploit vulnerabilities in the system to access all knowledge stored within it, thereby compromising the privacy of knowledge providers. Specifically, attackers may obtain sensitive confidential data contributed by knowledge providers, which poses significant risks to privacy and data integrity.

2) KDC Trust Assumptions: We assume that the Key Distribution Center (KDC) operates as a fully trusted authority responsible for issuing subkeys exclusively to authenticated users. It runs within a tamper-resistant and attestable environment (e.g., HSM or TEE), protecting its internal state, including the master secret key msk, from any unauthorized access or modification. The KDC deterministically executes the subkey-generation algorithm upon receiving valid query identifiers, ensuring correctness and preventing key forgery or duplication. All communication with clients is secured via authenticated and encrypted channels (e.g., mutual TLS). Other system components, such as retrieval nodes, are denied access to the secure enclave and cannot obtain msk, forge subkeys, or decrypt unauthorized inner-product results.

B. Design Goal

To address the privacy issues in RAG systems, we aim to design a comprehensive privacy-preserving architecture with the following goals:

- *Encrypted knowledge storage:* Ensure that knowledge remains encrypted, preventing unauthorized access to unencrypted copies and preserving data confidentiality.
- *Confidential knowledge computing:* Support secure computation on encrypted data, including retrieval of knowledge and its use in generating outputs by LLMs, thereby maintaining privacy throughout the process.

V. Design of CipheRAG

In this section, we provide the detailed design of CipheRAG.

A. Overview of CipheRAG

The overview of the CipheRAG is illustrated in Fig. 2. The system features three main phases: the storage phase, the retrieval phase, and the generation phase. Solid lines depict the end-to-end workflow from query to model generation, whereas dotted lines indicate on-chain and off-chain storage; the latter will be described in detail in Fig. 3.

In the **storage phase**, knowledge is stored through a dual approach based on blockchain [48] encompassing both on-chain and off-chain mechanisms. On-chain storage implements the ALSH-based encryption, facilitating secure and efficient knowledge retrieval. Concurrently, an inner product encrypted version of the original data is maintained off-chain. This dual storage methodology ensures privacy protection and efficient access to knowledge.

The **retrieval phase** begins with the processing of a query. A subkey generation mechanism is utilized, which, together with ALSH transformations, facilitates maximum inner product search within the knowledge-based blockchain. This process enables CipheRAG to identify and retrieve the most relevant knowledge from the stored data, ensuring that the LLM can access the most pertinent information for generating responses. The retrieval procedure is made verifiable by recording the retrieval inputs/outputs on-chain. The actual similarity computation can be executed by authorized off-chain nodes (e.g., oracles) and the resulting Top-k identifiers are committed to the contract state for public auditability. Clients submit their query q to the contract, which emits a `retrieve` event. A set of trusted oracle nodes then retrieve over the encrypted index on-chain and write the Top-k list of ciphertext identifiers back into the contract state. This on-chain execution guarantees immutability and public

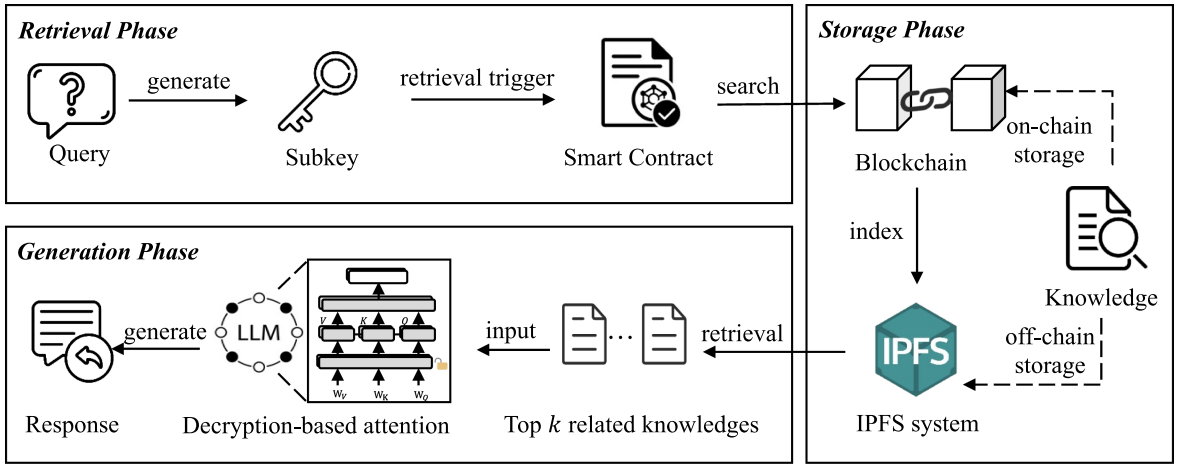

Fig. 2. Overview of CipheRAG. The solid lines illustrate the end-to-end retrieval process, starting from the user's query and culminating in response generation through the integration of decrypted knowledge. In contrast, the dotted lines depict the storage workflow.

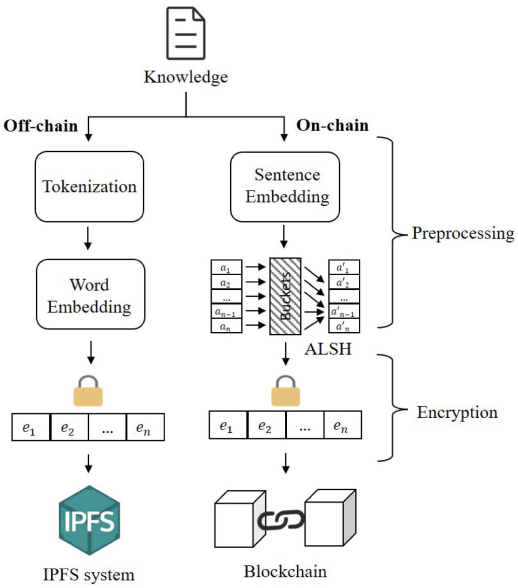

Fig. 3. The on-chain and off-chain storage of CipheRAG.

auditability of the exact retrieval result used for subsequent generation. All on-chain data remains encrypted, ensuring that the underlying information is never directly exposed. According to the security definition of IPFE, only the inner product values are revealed during computation, while the encrypted knowledge and queries themselves remain confidential.

In the **generation phase**, the retrieved knowledge is fed into a decryption-based attention layer that decrypts the knowledge to generate the final response. This phase integrates the retrieved knowledge into the LLM, enabling it to produce coherent and contextually appropriate answers. The modular framework of CipheRAG ensures both efficient knowledge retrieval and secure data handling, thereby enhancing the capabilities of LLMs in applications that require sensitive information processing.

As illustrated in Fig. 3, the CipheRAG effectively integrates blockchain and the InterPlanetary File System (IPFS) [49] to manage both on-chain and off-chain knowledge storage.

- **Efficient retrieval enabled on-chain storage**, encrypted hashes are generated from sentence embeddings using ALSH, resulting in a lightweight storage solution. These hashes enable efficient retrieval on the blockchain. Once the on-chain retrieval identifies the relevant content, it links to the corresponding off-chain data.
- **Integrity generation enabled off-chain storage**, tokenized data is processed into word embeddings, encrypted, and stored in IPFS. This approach ensures data integrity and security while enabling the LLM to generate responses using the complete off-chain content.

This dual system achieves lightweight and efficient on-chain retrieval, with secure LLM generation using off-chain data.

B. Privacy-Preserving Storage With Ada-IPFE

To preserve privacy in CipheRAG, knowledge embeddings are encrypted using the Ada-IPFE scheme, a concrete instantiation of the general class of IPFE methods. IPFE schemes are particularly well-suited for retrieval-augmented generation systems because they support efficient inner product similarity search, the core retrieval operation, directly on encrypted data. By employing Ada-IPFE, CipheRAG achieves strong privacy protection with significantly lower computational overhead than fully homomorphic encryption and without the accuracy loss associated with DP.

In Ada-IPFE, both ciphertexts and decryption keys are associated with vectors, and decryption yields only their inner product, exposing no other information. Specifically, a knowledge vector $\mathbf{x} \in \mathbb{Z}_N^n$ is encrypted to a ciphertext ct; subsequently, an authorized party can obtain $\langle \mathbf{x}, \mathbf{y} \rangle$ for a query vector $\mathbf{y}$, without learning anything beyond that inner product.

During the **setup phase**, the authority generates an RSA modulus $N = \tilde{p} \cdot \tilde{q}$, where $\tilde{p}$ and $\tilde{q}$ are safe primes, samples a random generator $g \in \mathbb{Z}_{N^2}^*$, and selects a master secret vector

$\mathbf{s} \in \mathbb{Z}_N^n$. The public elements are computed as $\eta_i = g^{s_i} \bmod N^2$ for $i \in [n]$. The master public and secret keys are

$$\text{mpk} = (g, \{\eta_i\}_{i=1}^n, N), \quad \text{msk} = \mathbf{s}.$$

In the **encryption phase**, the knowledge vector $\mathbf{x}$ is encrypted as follows. Choose fresh random values $r \in \mathbb{Z}_N$, $a, b, c \in \mathbb{Z}_N$, and let $\sigma = \text{pk}_2^c \bmod N$ where pk_2 is a public parameter associated with the query. The ciphertext is constructed as:

$$\begin{aligned} \sigma &= \text{pk}_2^c \bmod N, \\ \text{ct}_1 &= g^r \bmod N^2, \text{ct}_2 = g^b \bmod N^2, \\ \text{ct}_3 &= (c \cdot a - b) \bmod N, \text{ct}_4 = \text{pk}_1^r \bmod N^2, \\ \text{ct}_5 &= \left\{ \text{ct}_i = h_i^r (1 + N\sigma x_i) \bmod N^2 \right\}_{i \in [1,n]}. \end{aligned} \tag{12}$$

The resulting ciphertext is $\text{ct} = (\text{ct}_0, \text{ct}_1, \text{ct}_2, \text{ct}_3, \text{ct}_4, \text{ct}_5)$.

This design provides two important guarantees: (1) A functional decryption key $\text{sk}_\mathbf{y}$ is derived using the master secret vector and a chosen attribute vector $\mathbf{y} \in \mathbb{Z}_N^n$, enabling computation of $\langle \mathbf{x}, \mathbf{y} \rangle$ and nothing else about $\mathbf{x}$. (2) By encrypting knowledge vectors and enabling their use through weight matrices within neural network models, both privacy and on-chain data availability are achieved.

C. Efficient Retrieval Enabled Preprocessing

In CipheRAG, knowledge is processed and stored both on-chain and off-chain within the blockchain infrastructure. On-chain knowledge is primarily used for quick retrieval, whereas off-chain knowledge is used for generation. Consequently, different processing methods are applied.

As shown in Fig. 3, we implement distinct embedding strategies for knowledge representation in the off-chain and on-chain phases. For knowledge stored off-chain, we adopt the standard LLM-based embedding process, which includes tokenization followed by contextual word embedding using a pre-trained language model [50]. For the on-chain phase, we follow a structured pipeline to ensure efficient and privacy-preserving retrieval:

- *Sentence Embedding:* Each knowledge document is encoded into a 384-dimensional vector using Sentence-BERT [51].
- *ALSH Transformation:* To enable inner product-based nearest neighbor search, we apply the ALSH scheme proposed in [24]. Specifically, each knowledge vector $\mathbf{x} \in \mathbb{R}^{384}$ is transformed into $P(\mathbf{x})$.
- *Hashing and Quantization:* After ALSH transformation, we apply random projection with $K = 128$ hyperplanes to binarize each vector into a 128-bit hash code. This value of K balances retrieval speed and collision probability in our deployment:
- *Encryption:* The binary hash codes are encrypted using the IPFE scheme, which allows exact inner product evaluation without exposing either the plaintext vector or the projection direction.
- *On-chain Storage:* The IPFE-encrypted hash codes are uploaded to the blockchain.

1) Alsh: Efficient knowledge retrieval is crucial, as entities with subkeys frequently need to access the blockchain to search for knowledge through inner product computations. To enable efficient knowledge retrieval, we transform the sentence embeddings using L2-norm-based LSH, which maps data points into buckets in a hash table, facilitating fast and approximate nearest-neighbor searches [24].

Specifically, MIPS is utilized for a given knowledge v and query q to identify similarities in the inner product with other knowledge within a specified range. To maintain generality, assume a constant $I < 1$ exists such that every vector $v_i \in S$ satisfies the condition $\|v_i\|_2 \leq I$. Otherwise, we can rescale the vectors by applying the following transformation:

$$S(v) = \frac{V}{F} \cdot x, \quad \text{where} \quad F = \max_{v_i \in S} \|v_i\|_2, \tag{13}$$

where F represents the largest L2-norm among all the set S vectors. The transformation $S(x)$ adjusts the vector x to ensure it complies with the norm constraint defined by I.

However, direct computation of all relevant inner products for this purpose is impractical. To address this, we implement the ALSH [52] transform, effectively transforming the MIPS challenge into a nearest-neighbor search problem under Euclidean distance. We define the P transformation and Q transformation as follows:

$$P(v) = \left[v; \|v\|_2^2; \|v\|_2^{2^2}, \ldots, \|v\|_2^{2^m} \right], \tag{14}$$

$$Q(q) = \left[q; \frac{1}{2}; \frac{1}{2}, \ldots, \frac{1}{2} \right], \tag{15}$$

Then, the following equation can be obtained,

$$\begin{aligned} \|P(v) - Q(q)\|_2^2 &= (1 + m/4) - 2q^T v + \|v\|_2^{2^{m+1}} \\ &= (v - q)^2 + \left(\frac{1}{2} - \|v\|_2^2 \right)^2 + \left(\frac{1}{2} - \|v\|_2^{2^{m+1}} \right)^2, \end{aligned} \tag{16}$$

where the first term is constant with respect to v (for a fixed q). For the third term, given that each element of v is less than 1, it converges to 0 exponentially as m increases. This leaves us predominantly with the second term, which encompasses the required inner product.

$$\arg\max_v q^T v \approx \arg\min_v \|Q(q) - P(v)\|_2. \tag{17}$$

By finding the maximum inner product, we seek the minimum distance, transforming the original problem into a Euclidean nearest-neighbor search between $P(v)$ and $Q(q)$.

Once the ALSH transformation maps both the knowledge vector v and the query vector q into a common Euclidean space via $P(v)$ and $Q(q)$, we further discretize the transformed space by applying random hyperplane hashing:

$$h(v) = \text{sign}\left(r^T P(v) \right), \quad r \sim \mathcal{N}(0, I), \tag{18}$$

$$h(q) = \text{sign}\left(r^T Q(q) \right). \tag{19}$$

Each random projection r yields one bit of the binary hash code, and the concatenation of K such bits forms the final signature:

$$H(v) = [h_1(v), h_2(v), \ldots, h_K(v)] \in \{-1, +1\}^K, \quad (20)$$

$$H(q) = [h_1(q), h_2(q), \ldots, h_K(q)] \in \{-1, +1\}^K. \quad (21)$$

To preserve privacy, we encrypt each hashed signature $H(v)$ using IPFE. Let $H(v)$ be treated as a binary vector in $\mathbb{R}^K$ where entries are in $\{-1, +1\}$, we compute: $\texttt{Enc}(H(v))$. These encrypted hash codes are stored on-chain as secure bucket representations, without revealing the original vector or its transformed form. At query time, the client computes the cleartext signature $H(q)$ based on the public projection vectors $\{r_i\}$ and constructs a decryption key using the IPFE scheme: $\text{sk}_q = \texttt{KeyGen}(H(q))$. The match between a stored bucket and the query is evaluated by decrypting the inner product: $s = \texttt{Dec}(\texttt{Enc}(H(v)), \text{sk}_q)$. Since $H(v), H(q) \in \{-1, +1\}^K$, the inner product s represents the number of matching bits (up to an affine transformation), i.e., $s = \sum_{i=1}^{K} H_i(v) \cdot H_i(q)$. After computing the similarity scores s for all candidate vectors, the top-k highest similarity vectors are selected for retrieval.

2) Integrity-Verifiable On-Chain Retrieval: To prevent a malicious node from omitting or substituting retrieval results, we implement the entire Top-k selection pipeline as an on-chain smart-contract call. The client computes the query hash signature $\mathbf{H}(\mathbf{q})$ off-chain and locally generates the corresponding IPFE decryption key Key_q. It then invokes the contract's function: `retrieve(queryId, sk`$_q$`)` which, in a single atomic transaction, loads each encrypted bucket signature $\mathsf{Enc}(H(v_i))$ from the on-chain index and computes the inner product score s_i via IPFE decryption. These scores are used to determine the Top-k most similar entries. The resulting list of Top-k ciphertext identifiers is written to the contract's `retrievals[queryId]` mapping and emitted via a `Retrieved` event. To provide auditability, every hash signature decryption operation is immutably recorded on-chain. Any unauthorized modification, omission, or substitution is immediately detectable, and external auditors can independently verify the correctness and consistency of the retrieval results by invoking the following query:

`getRetrieval(queryId)` $\rightarrow$ `bytes32[] cList.`

This independent recomputation and verification capability ensures the verifiability, transparency, and accountability of the entire retrieval pipeline, thereby safeguarding against integrity attacks and malicious manipulations.

D. Generation With Decryption-Based Self-Attention Module

In this subsection, we introduce a decryption-based attention layer that integrates the decryption of off-chain encrypted knowledge embedding $\mathbf{x}$ directly into the attention mechanism of the LLMs. Instead of using a separate linear transformation layer for decryption, we leverage the parameters of the Query-Key-Value (QKV) linear operations [26] within the attention layers to generate the decryption keys. This approach ensures the secure integration of encrypted knowledge into the model's computations, naturally embedding the decryption process into the LLM architecture.

Algorithm 1: Encrypted Hash-Based Knowledge Retrieval.

Input: Query q, Master secret key msk, Knowledge Base $\mathcal{K} = \{\mathbf{v}_i\}_{i=1}^n$.
Output: Top-k relevant knowledge items.

Step I: Query preprocessing (off-chain)
Compute sentence embedding $\mathbf{q}$ from query q;
Apply ALSH Q-transformation: $Q(\mathbf{q})$;
Compute hash signature $H(q)$;
Generate decryption key: $\text{Key}_q = \texttt{KeyGen}(H(q))$;
Step II: On-chain encrypted bucket matching
Invoke smart contract to perform bucket comparison in encrypted space:
`ids = retrieveContract(queryId,` sk$_q$`);`
for *each encrypted hash code* $\texttt{Enc}(H(v_i))$ *stored on-chain* **do**
  Compute similarity score by decrypting inner product:
  $s_i = \texttt{Dec}(\texttt{Enc}(H(v_i)), \text{sk}_q)$;
end
Step III: Top-k retrieval and result registration
Sort scores s_i in descending order to rank relevance;
Select the top-k ciphertext identifiers $\{id_{(1)}, \ldots, id_{(k)}\}$;
Store Top-k results on-chain in `retrievals[queryId];`
Emit `Retrieved` event for public auditability;
return $\{id_{(1)}, \ldots, id_{(k)}\}$;

1) Integration Into the Attention Mechanism: The attention mechanism in LLMs projects the input embedding $\mathbf{x} \in \mathbb{R}^{n\times 1}$ and positional encoding $\mathbf{pe} \in \mathbb{R}^{n\times 1}$ into Query, Key, and Value vectors via learned weight matrices:

$$\begin{bmatrix} \mathbf{Q} \\ \mathbf{K} \\ \mathbf{V} \end{bmatrix} = \mathbf{W}(\mathbf{x} + \mathbf{pe}) = \mathbf{W}\mathbf{x} + \mathbf{W}\mathbf{pe}, \quad (22)$$

where $\mathbf{W} \in \mathbb{R}^{3d'\times n}$ is formed by vertically stacking the Q, K, and V weights. In our privacy-preserving design, the embedding $\mathbf{x}$ is stored in encrypted form using Ada-IPFE.

2) Decryption Key Generation: Given the master secret key msk from the Ada-IPFE setup, we generate a decryption subkey for each projection direction. Specifically, for each row $\mathbf{w}_i$ of the projection matrix $\mathbf{W}$, we compute the corresponding functional key as:

$$\mathbf{sk_i} = \texttt{KeyGen}(msk, \mathbf{w}_i) \quad (23)$$

where `KeyGen` denotes the Ada-IPFE key generation algorithm.

3) Decryption With the Attention Layer: To recover the required projections&, we decrypt the encrypted embedding $\mathbf{x}$ with each subkey $\mathbf{sk}_i$ via the Ada-IPFE decryption function:

$$\mathbf{x}^\top \mathbf{w_i}^\top = \texttt{Dec}(\mathbf{sk}_i, \texttt{Enc}(\mathbf{x})) \quad (24)$$

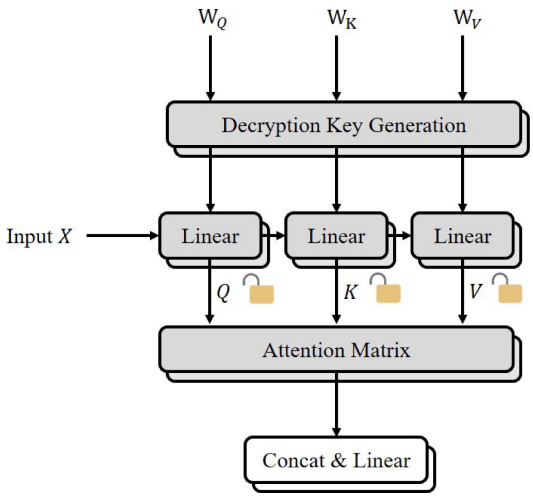

Fig. 4. The decryption-based attention module. $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are the query, key and value vectors.

where d_i is the plaintext inner product $\mathbf{w}_i\mathbf{x}$. This procedure is repeated for all rows of $\mathbf{W}$, yielding the projected Q, K, V vectors in plaintext. The complete projection is thus reconstructed as:

$$\mathbf{W}\mathbf{x} = \begin{bmatrix} \mathbf{w}_1\mathbf{x} \\ \mathbf{w}_2\mathbf{x} \\ \vdots \\ \mathbf{w}_n\mathbf{x} \end{bmatrix} = \begin{bmatrix} \mathbf{x}^\top\mathbf{w}_1^\top \\ \mathbf{x}^\top\mathbf{w}_2^\top \\ \vdots \\ \mathbf{x}^\top\mathbf{w}_n^\top \end{bmatrix}, \tag{25}$$

where each d_i is obtained via Ada-IPFE decryption.

4) Computing the Attention Output: With the decrypted Query, Key, and Value matrices, the subsequent computation follows the standard self-attention mechanism:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}, \tag{26}$$

as shown in Fig. 4. This approach ensures that all projections are performed over encrypted data, with decryption securely realized through Ada-IPFE functional keys.

VI. Property Analysis

In this section, we provide a theoretical analysis of the security of CipheRAG.

A. Adaptive Security of the Ada-IPFE Scheme

1) Security Notation: We formalize the simulation-based adaptive security experiment for Ada-IPFE as follows.

Definition 1 (Challenge-response Simulation-Based Security for Ada-IPFE): The Ada-IPFE scheme is adaptively simulation-based secure (in the sense of IND-FE) if for any PPT adversary $\mathcal{A}$, the advantage in the following experiment is negligible:

Setup: The challenger runs the setup algorithm and gives the public parameters to $\mathcal{A}$.

Key Queries: $\mathcal{A}$ may adaptively query secret keys $\text{sk}_\mathbf{y}$ for any legal $\mathbf{y}$.

Algorithm 2: Decryption-Based Attention Generation.

Input: Let $\mathcal{E} = \{\mathbf{x}_i\}_{i=1}^k$ denote the retrieved top-k encrypted embeddings, master secret key msk, and transformer weight matrix $\mathbf{W}$.

Output: Decrypted embedding representations after attention $\mathbf{H}$.

Step I: Decryption Key Generation
for *each row vector* $\mathbf{w}_i$ *of* $\mathbf{W}$ **do**
  Generate decryption subkey $\mathbf{sk}_i$;
end
Step II: Decryption
for *each encrypted embedding* $\mathbf{x}_i \in \mathcal{E}$ **do**
  Decrypt the inner product to obtain $d_i = \mathbf{x}_i^\top \mathbf{w}_i^\top$;
end
Step III: QKV Matrix Projection
Use the decrypted values $\{d_i\}$ to compute the Query, Key, and Value projections, forming $\mathbf{Q}, \mathbf{K}, \mathbf{V}$;
Step IV: Attention Mechanism
Compute $\mathbf{H} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$;
return Decrypted embedding representations $\mathbf{H}$.

Challenge: $\mathcal{A}$ submits $(\mathbf{z}^{[0]}, \mathbf{z}^{[1]})$, and receives the encryption of $\mathbf{z}^{[b]}$ for random b.

Restriction: All key queries must satisfy $\langle \mathbf{z}^{[0]} - \mathbf{z}^{[1]}, \mathbf{y} \rangle = 0$.

Guess: $\mathcal{A}$ outputs a bit b'; its advantage is $|\Pr[b' = b] - \frac{1}{2}|$. The scheme is secure if this advantage is negligible.

2) Game-Based Analysis: We now prove that the Ada-IPFE construction achieves the above challenge-response simulation-based security. The proof proceeds via a sequence of games.

Theorem 1: The Ada-IPFE scheme is adaptively secure under the DCR assumption.

Proof: Let $\mathcal{A}$ be a probabilistic polynomial-time adversary interacting with the scheme. We describe a sequence of games to analyze the advantage of $\mathcal{A}$.

Game 0: The challenger runs the setup algorithm to produce the public parameters $\text{pp} = (g, \{\eta_i\}_{i=1}^n, N, Z)$, which are given to $\mathcal{A}$, and a master secret key $\mathbf{s} \in \mathbb{Z}_N^n$, which remains private. During the challenge phase, $\mathcal{A}$ is permitted to specify a pair of vectors $\mathbf{z}^{[0]}, \mathbf{z}^{[1]} \in \mathbb{Z}_N^n$ such that $\|\mathbf{z}^{[0]}\|, \|\mathbf{z}^{[1]}\| \leq Z$, where Z is the allowed norm bound.

The challenger selects a uniform random bit $b \in \{0, 1\}$ and responds with the encryption of $\mathbf{z}^{[b]}$, denoted ct^*.

At any point prior to or after the challenge, $\mathcal{A}$ may issue secret key requests for function vectors $\mathbf{y} \in \mathbb{Z}_N^n$, subject to the restriction that all queries satisfy the orthogonality property:

$$\left\langle \mathbf{z}^{[0]} - \mathbf{z}^{[1]}, \mathbf{y} \right\rangle = 0, \tag{27}$$

with the inner product computed over the integers. This condition precludes trivial separation of the challenge bit via function queries aligned with the difference of the target messages.

Ultimately, $\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$ for the challenge bit. The experiment is said to be won if $b' = b$. The adversary's

distinguishing advantage is defined as

$$\left|\Pr[b' = b] - \frac{1}{2}\right|. \tag{28}$$

Game 1: The procedure follows Game 0, except that the encryption of the challenge vector $\mathbf{z}^{[b]}$ is randomized at its core group element. Specifically, the challenger samples a random $\kappa_0 \leftarrow \mathbb{Z}_N^*$, and computes $\kappa = \kappa_0^N \bmod N^2$. Then, the key ciphertext components are formed as:

$$\mathsf{ct}_1^* = \kappa^2 \bmod N^2, \mathsf{ct}_4^* = (\mathsf{ct}_1^*)^\alpha \bmod N^2,$$
$$\mathsf{ct}_{5,i}^* = (\mathsf{ct}_1^*)^{s_i} \left(1 + N\sigma z_i^{[b]}\right) \bmod N^2, \quad \forall i \in [n], \tag{29}$$

where α and s_i are as in msk, and σ is fresh randomness.

All other parts of the ciphertext (such as ct_0^*, ct_2^*, ct_3^*) are computed as in the real scheme. Since ct_1^* is now uniform in the $2N$-th residue subgroup, the resulting ciphertext has distribution statistically close to Game 0:

$$|\Pr[S_1] - \Pr[S_0]| \leq \frac{1}{2^\lambda}. \tag{30}$$

Game 2: In this game, we further alter the encryption: now the core element κ used for ct_1^* is not of the special form κ_0^N, but is simply chosen uniformly at random from $\mathbb{Z}_{N^2}^*$: $\mathsf{ct}_1^* = \kappa^2 \bmod N^2$, where $\kappa \leftarrow \mathbb{Z}_{N^2}^*$. The remaining ciphertext components are generated just as in Game 1, using this κ.

Under the DCR assumption, the challenge ciphertext distribution in Game 2 is computationally indistinguishable from Game 1:

$$|\Pr[S_2] - \Pr[S_1]| \leq \mathrm{Adv}_{\mathcal{A}}^{\mathrm{DCR}}(\lambda). \tag{31}$$

Game 3: In this game, we show that the adversary's view, including all previously queried keys and the simulated challenge ciphertext, is statistically independent of the challenge bit $b \in \{0, 1\}$.

For every $i \in [n]$, the i-th ciphertext component is:

$$\mathrm{ct}_i^{(b)} = \eta_i^p (1 + \sigma N) \left(z_i^{(b)} + a_p s_i\right) \bmod N^2, \tag{32}$$

where p is a fresh random exponent, σ and a_p are uniformly random, and s_i is the i-th entry of the secret vector $\mathbf{s}$. The challenge message is $\mathbf{z}^{(b)} \in \mathbb{Z}^n$, indexed by the hidden bit b.

To analyze the statistical hiding, we define:

$$\boldsymbol{\delta} = \frac{1}{d}\left(\mathbf{z}^{(1)} - \mathbf{z}^{(0)}\right),$$
$$d = \gcd\left(z_1^{(1)} - z_1^{(0)}, \ldots, z_n^{(1)} - z_n^{(0)}\right),$$
$$\mathcal{L}_z = \{\mathbf{y} \in \mathbb{Z}^n \mid \langle \mathbf{z}, \mathbf{y}\rangle = 0\}. \tag{33}$$

Any legal key query $\mathbf{y}$ by the adversary must satisfy $\langle \mathbf{z}, \mathbf{y}\rangle = 0$, so all possible queries are confined to the lattice $\mathcal{L}_x$.

We note that, under the restriction $\langle \mathbf{z}^{(1)} - \mathbf{z}^{(0)}, \mathbf{y}\rangle = 0$, all valid key queries $\mathbf{y}$ lie in the nullspace of the difference vector, which forms an $(n-1)$-dimensional subspace of $\mathbb{Z}_N^n$. We denote a basis for this subspace by $\{\mathbf{u}_1, \ldots, \mathbf{u}_{n-1}\}$, and stack them as the rows of a matrix $\mathcal{U} \in \mathbb{Z}_N^{(n-1)\times n}$.

Given this structure, the information that the adversary can obtain about the secret $\mathbf{s}$ through any valid queries is determined solely by the vector $\mathcal{U} \cdot \mathbf{s}^T \in \mathbb{Z}_N^{n-1}$. Note that for any ciphertext component $\boldsymbol{\delta}^{(b)} = \mathbf{z}^{(b)} + a_p \mathbf{s}$ (with a_p uniformly random), we have

$$\mathcal{U} \cdot \left(\boldsymbol{\delta}^{(b)}\right)^T \bmod N = \mathcal{U} \cdot \left(\mathbf{z}^{(b)}\right)^T + a_p \cdot \mathcal{U} \cdot \mathbf{s}^T \bmod N. \tag{34}$$

Since a_p is uniformly random and invertible in $\mathbb{Z}_N$, for any fixed $\mathcal{U} \cdot \mathbf{s}^T$ the adversary's view of $\boldsymbol{\delta}^{(b)}$ is statistically independent of b.

Consequently, the conditional distribution of $\boldsymbol{\delta}^{(b)}$ given all obtainable information is uniform for either value of b, and no information about the challenge bit can be learned.

Finally, for any efficiently computable function f, the statistical distance between the adversary's view when $b = 0$ and $b = 1$ is at most $\frac{1}{2^{\lambda-1}}$. Together with the DCR assumption, we have:

$$\left|\Pr[S_0] - \frac{1}{2}\right| \leq \mathrm{Adv}_{\mathcal{A}}^{\mathrm{DCR}}(\lambda) + \frac{1}{2^{\lambda-1}}. \tag{35}$$

This concludes that the adversary's distinguishing advantage is negligible, even after making arbitrary adaptive key queries. Thus, under the DCR assumption, the Ada-IPFE scheme is adaptively secure.

B. Correctness of the Ada-IPFE Scheme

We demonstrate the correctness of the proposed Ada-IPFE scheme, namely that the decryption of a ciphertext under an honestly generated functional key always yields the correct inner product $\langle \mathbf{z}, \mathbf{y}\rangle$ over $\mathbb{Z}_N$.

Theorem 2: The Ada-IPFE scheme instantiated above is correct: for all message vectors $\mathbf{z}$ and key vectors $\mathbf{y}$, the decryption procedure yields $\tau_2 = \langle \mathbf{z}, \mathbf{y}\rangle$.

Proof: Suppose the encryption and decryption algorithms are executed as specified before. Let the ciphertext of $\mathbf{z}$ be $ct = (\mathrm{ct}_0, \mathrm{ct}_1, \mathrm{ct}_2, \mathrm{ct}_3, \mathrm{ct}_4, \{\mathrm{ct}_i\}_{i=1}^n)$ and the decryption key for $\mathbf{y}$ be $\mathrm{sk}_{\mathbf{y}} = (\beta, s_{\mathbf{y}})$. According to the scheme:

$$\tau_1 = ((\mathrm{ct}_2 \cdot g^{\mathrm{ct}_3})^{\beta \cdot a^{-1} \bmod N} \bmod N^2) \bmod N,$$
$$\tau_2 = \frac{\left(\mathrm{ct}_1^{(\beta - s_{\mathbf{y}}) \bmod N} \mathrm{ct}_4 \prod_{i=1}^n \mathrm{ct}_i^{y_i} \tau_1^{-1} - 1\right) \bmod N^2}{N}. \tag{36}$$

Expanding the above terms, we note that $\mathrm{ct}_i = \eta_i^r (1 + N\sigma z_i) \bmod N^2$ and $\eta_i = g^{s_i}$, so

$$\prod_{i=1}^n \mathrm{ct}_i^{y_i} = g^{r\langle \mathbf{s}, \mathbf{y}\rangle} \cdot \prod_{i=1}^n (1 + N\sigma z_i)^{y_i} \bmod N^2. \tag{37}$$

By appropriately combining all relevant terms in (12) and exploiting the scheme's homomorphic properties, the decryption process cancels all blinding terms and randomizers. Thus, we

obtain:

$$\tau_2 = \frac{(1 + N\langle \mathbf{z}, \mathbf{y}\rangle - 1) \bmod N^2}{N} = \langle \mathbf{z}, \mathbf{y}\rangle. \tag{38}$$

Furthermore, by the norm bound assumption $\|\mathbf{z}\| \le Z, \|\mathbf{y}\| \le Y, ZY < N$, we guarantee that $\langle \mathbf{z}, \mathbf{y}\rangle < N$, so no modular overflow occurs. The Ada-IPFE scheme thus correctly outputs the intended inner product for all valid inputs and keys. ■

C. Standard Consistency Constraint of Ada-IPFE

The standard consistency constraint requires that a ciphertext encrypted under a public key $\mathsf{pk}_{\mathbf{y}}$ can only be correctly decrypted by the matching secret key $\mathsf{sk}_{\mathbf{y}}$, and not by a secret key generated for any other vector $\hat{\mathbf{y}} \neq \mathbf{y}$.

Theorem 3: The Ada-IPFE scheme satisfies the standard consistency constraint.

Proof: Suppose, for contradiction, that there exist two distinct decryption keys $\mathsf{sk}_{\mathbf{y}}$ and $\mathsf{sk}_{\hat{\mathbf{y}}}$ (generated for vectors $\mathbf{y}$ and $\hat{\mathbf{y}}$, respectively, with $\mathbf{y} \neq \hat{\mathbf{y}}$), such that a ciphertext $\mathsf{ct}_{\mathbf{z}}$ encrypted under $\mathsf{pk}_{\mathbf{y}}$ can be decrypted by $\mathsf{sk}_{\hat{\mathbf{y}}}$ to recover $\langle \mathbf{z}, \hat{\mathbf{y}}\rangle$. Recall the key generation process: for vector $\mathbf{y}$,

$$\mathsf{sk}_{\mathbf{y}} = (\beta, s_{\mathbf{y}}), \quad s_{\mathbf{y}} = \langle \mathbf{s}, \mathbf{y}\rangle + \alpha + \beta, \tag{39}$$

with public key $\mathsf{pk}_{\mathbf{y}} = (g^{\alpha} \bmod N^2, g^{\beta} \bmod N^2)$, and for $\hat{\mathbf{y}}$,

$$\mathsf{sk}_{\hat{\mathbf{y}}} = (\hat{\beta}, s_{\hat{\mathbf{y}}}), \quad s_{\hat{\mathbf{y}}} = \langle \mathbf{s}, \hat{\mathbf{y}}\rangle + \hat{\alpha} + \hat{\beta}, \tag{40}$$

with public key $\mathsf{pk}_{\hat{\mathbf{y}}} = (g^{\hat{\alpha}} \bmod N^2, g^{\hat{\beta}} \bmod N^2)$, where $(\hat{\alpha}, \hat{\beta}) \neq (\alpha, \beta)$. The ciphertext for $\mathbf{z}$ under $\mathsf{pk}_{\mathbf{y}}$ is constructed as (12).

If $\mathsf{sk}_{\hat{\mathbf{y}}}$ could decrypt $\mathsf{ct}_{\mathbf{z}}$, the decryption equations yield:

$$\hat{\tau}_1 = \left((\mathsf{ct}_2 \cdot g^{\mathsf{ct}_3})^{\hat{\beta}} \cdot \mathsf{ct}_0^{-\hat{\beta}} \bmod N^2\right)^{-1} \bmod N,$$
$$\hat{\tau}_2 = \frac{\left(\mathsf{ct}_1^{(\hat{\beta} - s_{\hat{\mathbf{y}}}) \bmod N} \, \mathsf{ct}_4 \prod_{i=1}^{n} \mathsf{ct}_i^{\hat{y}_i} \cdot \hat{\omega}_1^{-1} - 1\right) \bmod N^2}{N}. \tag{41}$$

Working out the algebra, it follows that for successful decryption, both $g^{\hat{\alpha}-\alpha} = 1$ and $g^{\beta-\hat{\beta}} = 1$ must hold modulo N^2, which (since g is a generator) implies $\hat{\alpha} = \alpha$ and $\hat{\beta} = \beta$. But by construction, $(\hat{\alpha}, \hat{\beta}) \neq (\alpha, \beta)$ for distinct key queries. Therefore, the ciphertext $\mathsf{ct}_{\mathbf{z}}$ under $\mathsf{pk}_{\mathbf{y}}$ can only be decrypted with $\mathsf{sk}_{\mathbf{y}}$, not with $\mathsf{sk}_{\hat{\mathbf{y}}}$. Hence, the Ada-IPFE scheme satisfies the standard consistency constraint. ■

D. Master Secret Key Hiding of Ada-IPFE

A crucial security requirement for functional encryption schemes is that the master secret key must remain information-theoretically hidden, even under adaptive key queries. In our Ada-IPFE construction, the msk is the vector $\mathbf{s} = (s_1, \ldots, s_n)$.

Theorem 4: The Ada-IPFE scheme achieves master secret key hiding.

Proof: Suppose an adversary $\mathcal{A}$ adaptively issues l function key queries for vectors $\mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(l)}$ and receives the corresponding secret keys

$$\mathsf{sk}_{\mathbf{y}^{(j)}} = \langle \mathbf{s}, \mathbf{y}^{(j)}\rangle + \alpha^{(j)} + \beta^{(j)}, \quad \forall j \in [l], \tag{42}$$

where $\alpha^{(j)}, \beta^{(j)}$ are randomly chosen in each query. To recover $\mathbf{s}$, $\mathcal{A}$ would need to solve the system of equations:

$$\begin{cases} \langle \mathbf{s}, \mathbf{y}^{(1)}\rangle + \alpha^{(1)} + \beta^{(1)} = \mathsf{sk}^{(1)}, \\ \vdots \\ \langle \mathbf{s}, \mathbf{y}^{(l)}\rangle + \alpha^{(l)} + \beta^{(l)} = \mathsf{sk}^{(l)}, \end{cases} \tag{43}$$

Given that each $(\alpha^{(j)}, \beta^{(j)})$ is independently and uniformly distributed, the probability of correctly recovering $\mathbf{s}$ is at most N^{-2l}, which is negligible for large modulus N. Brute-force matching of η_i is similarly infeasible. Thus, the Ada-IPFE scheme provides master secret key hiding. ■

E. Encrypted Vector Hiding of Ada-IPFE

Besides hiding the secret key, our scheme ensures that the encrypted message vector $\mathbf{z}$ cannot be recovered, even if the adversary obtains inner products $\langle \mathbf{z}, \mathbf{y}^{(1)}\rangle, \ldots, \langle \mathbf{z}, \mathbf{y}^{(n)}\rangle$ for linearly independent $\mathbf{y}^{(j)}$.

Theorem 5: The Ada-IPFE scheme achieves encrypted vector hiding.

Proof: By construction, each ciphertext of $\mathbf{z}$ can only be decrypted with respect to one specific function key. That is, for any $\mathbf{y}$, the decryption algorithm yields only $\langle \mathbf{z}, \mathbf{y}\rangle$; it does not expose additional linear relations that would enable recovery of the entire $\mathbf{z}$. The randomness in the encryption process ensures that no adversary, even given n linearly independent key queries, can solve for $\mathbf{z}$ except with negligible probability. Thus, Ada-IPFE achieves encrypted vector hiding. ■

VII. Experiments

We evaluate the performance of CipheRAG in terms of the retrieval and generation phases, respectively.

A. Experimental Setup

We use PyTorch to implement CipheRAG and evaluate it on an Intel(R) Core(TM) i9-14900F running at 2.00 GHz, 64 GB of RAM, and a Geforce RTX 4090 GPU.

In this experiment, we leverage Ganache and Truffle[1] tools for deploying and testing smart contracts while utilizing the IPFS for decentralized file storage. We chose the number of hash functions as $k = 512$ and the number of hash tables as $L = 15$. We set $m = 3$ and $U = 0.83$, following the parameters referenced by [52].

Datasets: We conduct experiments on two widely-used QA benchmarks: (1) **Amnesty QA** [2] is a fact-based dataset derived from Amnesty International's reports, focusing on grounded answers to global human rights issues; (2) **Natural Questions (NQ)** [53] consists of real user queries from Google Search,

[1] https://archive.trufflesuite.com/ganache/
[2] https://www.amnesty.org/en/research/

paired with answers grounded in Wikipedia passages. Both datasets are designed to evaluate the factuality and grounding of open-domain QA systems.

Metric: We used response relevancy, faithfulness, and correctness as metrics to measure the ability to generate retrieval augmentation for LLM.

- *Response Relevancy:* Evaluates the semantic alignment between the generated answer and the original question by computing the cosine similarity between their embeddings. Higher scores indicate more relevant and focused responses.
- *Faithfulness:* Measures factual consistency by assessing the proportion of generated claims that are supported by retrieved context. Scores range from 0 to 1, with higher values indicating stronger adherence to source evidence.
- *Correctness:* Assesses factual accuracy by comparing generated responses against reference answers using natural language inference. Precision, recall, and F1 scores are computed to reflect factual overlap.

The distributions are generated using kernel density estimation (KDE), which estimates the probability density function. KDE provides a smooth representation of data distribution, though the smoothing effect may extend beyond actual data boundaries. This boundary effect is expected and does not indicate the existence of data beyond these limits.

Model: We evaluate three representative LLMs: Llama3 [54] is Meta's state-of-the-art Transformer for efficient text and multimodal generation; Qwen2 [55] is Alibaba DAMO Academy's high-precision multilingual LLM; GPT-2 [56] is OpenAI's autoregressive model that set the benchmark for coherent text synthesis.

Baseline: This experiment presents an evaluation involving comparisons with advanced LLMs and state-of-the-art encryption-based LLM frameworks in the inference phase.

Retrieval Methods: Our evaluation compares three canonical RAG pipelines—Vanilla RAG, BGE-RAG, and FiD—against our ALSH scheme.

- *Vanilla RAG [1]:* employs BM25 sparse retrieval over a document corpus, followed by an autoregressive generator to synthesize answers.
- *BGE-RAG [57]:* uses BGE dense text embeddings with FAISS–based approximate nearest neighbor search to retrieve semantically relevant passages.
- *FiD [28]:* retrieves top-k passages via BM25 and fuses them jointly in a Fusion-in-Decoder architecture for enhanced answer generation.

Encryption-based Methods: We compare our approach with leading encryption-based methods, the Bumblebee and CipherGPT, to demonstrate superior data security.

- BumbleBee [20] is a secure two-party inference framework designed for large transformer models. It utilizes Oblivious Transfer (OT) [58] to enable secure model inference while preserving privacy. BumbleBee is particularly optimized for transformer-based architectures, ensuring that sensitive data remains encrypted throughout the computation process, but it incurs significant communication and computational overhead due to the multiple interactive rounds required between parties.
- CipherGPT [16] is a secure two-party inference framework tailored for GPT-like language models. It leverages FHE to enable privacy-preserving computations directly on encrypted data. CipherGPT excels in scenarios requiring high levels of data confidentiality but suffers from high computational complexity due to the extensive overhead associated with FHE-based operations.

TABLE III
AVERAGE RUNTIME IN GENERATION AND RETRIEVAL

Operation	IPFE		Ada-IPFE	
	Retrieval	Generation	Retrieval	Generation
Master Key generation	59.7 ms	1.91 s	89.6 ms	2.75 s
Encryption	62.3 ms	1.97 s	97.5 ms	3.06 s
Subkey generation	0.143 ms	4.06 ms	0.246 ms	6.67 ms
Discrete logarithms	17.2 ms	0.477 s	25.8 ms	0.802 s

TABLE IV
TOTAL GENERATION TIME (IN SECONDS)

Method	GPT2 (768,12)	Qwen2 (3584,16)	Llama3 (4096,32)
Baseline	0.73	4.54	5.81
ALSH-RAG	0.83	4.67	5.93
CipheRAG	6.92	88.53	121.27
BumbleBee [20]	162.3	870.2	1362.4
CipherGPT [16]	246.1	1134.8	1482.4

B. Experiments for the Efficiency

The experimental results in Table III provide a comparative analysis of the computational overheads associated with standard IPFE and Ada-IPFE. Overall, we observe that Ada-IPFE incurs a moderate increase in average runtime, typically between $1.2\times$ and $1.5\times$ the cost of IPFE, across both the generation and retrieval phases. This increase primarily results from the difference in constant factors inherent to the two cryptographic constructions; however, both schemes exhibit similar asymptotic complexity with respect to the vector dimension. In practice, the runtime gap does not grow significantly as the embedding dimension increases, since the dominant cost per operation remains linear in vector size for both schemes. The retrieval phase leverages L2-norm-based hashing to enable efficient similarity matching, while the generation phase prioritizes complete security and data fidelity, resulting in higher latency. These results confirm that, despite the additional computational cost of Ada-IPFE, its performance remains practical for latency-sensitive applications, such as secure healthcare dialogue, financial services, and interactive customer support. The modest overhead is a reasonable trade-off for the improved security guarantees and resistance to linearity attacks offered by Ada-IPFE.

Table IV compares our proposed approach with ALSH-RAG (non-encrypted), BumbleBee [20] (OT-based) and CipherGPT [16] (FHE-based) in terms of total generation time, while Table V highlights the QKV computation overhead within these frameworks. Both BumbleBee and CipherGPT represent significant advancements in secure model inference, leveraging

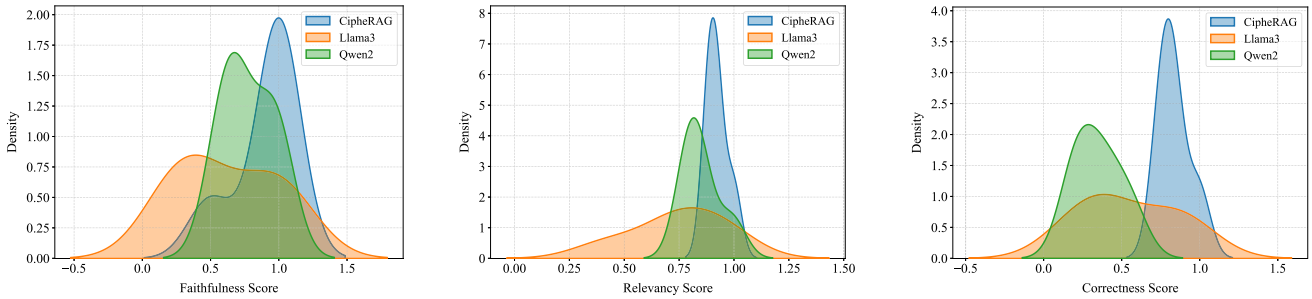

Fig. 5. Comparative analysis of CipheRAG, Llama3, and Qwen2 across faithfulness, relevancy, and correctness scores.

TABLE V
QKV CALCULATION TIME (IN SECONDS) WITHIN THE TOTAL GENERATION

Model	CipheRAG	BumbleBee [20]	CipherGPT [16]
GPT2 (768,12)	6.3	58.7	91.8
Qwen2 (3584,16)	81.2	612.5	672.3
Llama3 (4096,32)	120.5	1134.8	1188.2

strong cryptographic guarantees. However, these guarantees come at the cost of substantial computation and communication overhead, particularly during large-scale matrix multiplications involved in QKV calculations, which dominate the overall token generation process. This level of performance gain, achieved without relying on FHE or expensive secure protocols, positions CipheRAG as a practical alternative for real-time privacy-preserving inference in domains where efficiency and scalability are prioritized over theoretical cryptographic guarantees. For instance, edge-deployed intelligent agents or cloud-based AI APIs could integrate CipheRAG to deliver secure yet performant user experiences under moderate trust assumptions.

In contrast, our method adopts a partial encryption strategy that selectively decrypts data during the QKV vector calculation. This approach avoids performing all operations in the ciphertext domain, instead limiting cryptographic computations to vector-vector multiplications. By reducing the burden of fully encrypted matrix multiplications, our approach achieves significant efficiency gains. For example, generating one token with GPT2 requires 162.3 s in BumbleBee and 246.1 s in CipherGPT, whereas our method completes the same task in just 6.92 s, yielding approximately 23× and 35× speedups, respectively. Similarly, for GPT2, CipheRAG achieves a 23× and 36× speedup over BumbleBee and CipherGPT, respectively. For Qwen2, the acceleration is 10× and 12×, while for Llama3, the gains are 11× and 12× compared to BumbleBee and CipherGPT, respectively. The QKV computation presented in Table V demonstrates similar trends, with our method achieving up to 15× faster results compared to CipherGPT.

While our method does not aim to surpass the cryptographic strength of BumbleBee or CipherGPT, it demonstrates a conscious trade-off between privacy and efficiency. By partially encrypting only the critical stages and strategically decrypting during computational bottlenecks, we achieve a practical balance that dramatically reduces overhead while maintaining reasonable security guarantees. This makes our approach more suitable for real-world deployment where performance and usability are critical considerations.

C. Experiments for Retrieval Augmented Generation

Fig. 5 illustrates the performance of CipheRAG, Llama3, and Qwen2 across faithfulness, relevancy, and correctness scores. The distribution curves reveal notable differences in how these models handle various aspects of response quality. CipheRAG consistently outperforms the other models, as indicated by its sharp peaks around 1.0 for all three metrics, demonstrating faithful, relevant, and correct outputs. In contrast, Llama3 has broader distributions around lower values, reflecting significant variability and lower reliability in maintaining faithfulness, relevancy, and correctness. Qwen2 performs slightly better than Llama3 but still falls short of CipheRAG's consistency, particularly in terms of faithfulness and correctness, with scores generally closer to zero. Overall, CipheRAG provides the most reliable performance, with consistently high-quality outputs across all metrics.

Fig. 6 presents a side-by-side comparison of four RAG pipelines with Llama3, Vanilla-RAG [1], BGE [57], FiD [28], and our proposed CipheRAG, evaluated on Hit@10 recall, response relevancy, faithfulness, and answer correctness. The ALSH variant attains a Hit@10 of 0.96, nearly matching the BGE and FiD baselines (0.99 each) while vastly outperforming the Vanilla system (0.55). In terms of response relevancy, ALSH reaches 0.90, only marginally below BGE (0.94) and substantially above FiD (0.83) and Vanilla (0.87). For faithfulness and answer correctness, ALSH records 0.92 and 0.88 respectively, once again closely trailing BGE (0.94 and 0.89) and exceeding FiD (0.75 and 0.78). These results demonstrate that our ALSH-enabled encrypted retrieval preserves generative quality on par with state-of-the-art RAG methods, thereby validating its effectiveness for secure, high-fidelity knowledge integration. The consistently high scores across faithfulness, relevancy, and correctness metrics suggest that CipheRAG is well-equipped for knowledge-intensive generation scenarios, such as legal content drafting, clinical guideline summarization, and enterprise-level technical support. To maximize performance, it is essential to maintain a well-structured and semantically coherent encrypted knowledge base. This highlights the role of careful curation in enabling robust and accurate knowledge retrieval within

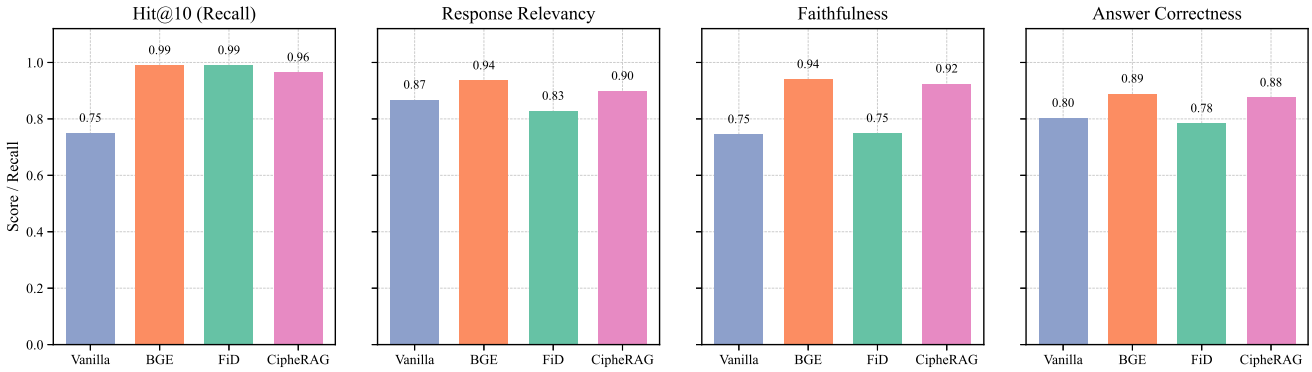

Fig. 6. Performance Comparison of CipheRAG and State-of-the-Art RAG Methods.

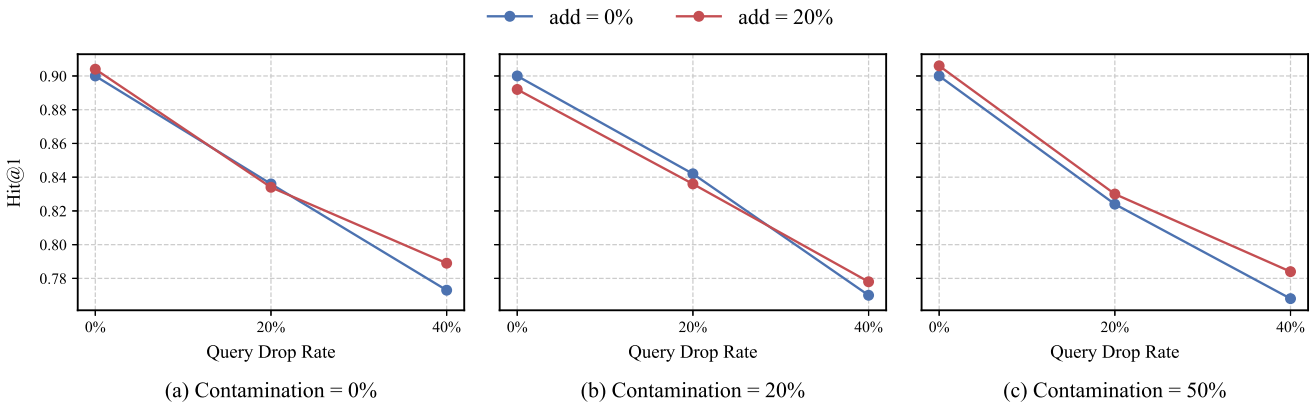

Fig. 7. Performance Comparison of CipheRAG and State-of-the-Art RAG Methods.

CipheRAG's framework, without introducing significant vulnerability to content misalignment.

D. Ablation Study

According to the recommendations of NIST [59], κ represents the symmetric security parameter, and ζ denotes the public-key security parameter. These parameters are specified within the ranges $\kappa \in \{80, 112, 128\}$ and $\zeta \in \{1024, 2048, 3072\}$, corresponding to legacy security (up to 2010), medium-term security (2011–2030), and long-term security (beyond 2030), respectively.

For this study, we selected $\kappa = 112$ and $\zeta = 2048$ to align with medium-term security requirements and to demonstrate why the Ada-IPFE method was chosen for the decryption-enabled attention layer rather than HE-based or OT-based approaches. The rationale for this choice is outlined as follows:

Firstly, with the Ada-IPFE method, knowledge providers can securely upload encrypted knowledge to a public database without requiring any interaction with the knowledge users or the LLM for decryption. This independence significantly reduces the complexity of the deployment process and enhances scalability. In contrast, both HE- and OT-based methods involve one or multiple rounds of communication between the knowledge provider and user to perform decryption or computation securely, introducing additional latency and potential bottlenecks.

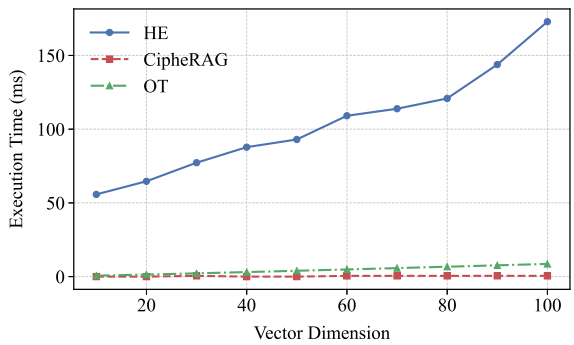

Fig. 8. The execution time of three encryption methods over vector dimension.

TABLE VI
THEORETICAL COMPLEXITY

Crypotography Method	Complexity
HE	$\mathbf{O(n^2 \log n)}$
OT	$\mathbf{O(nb^2 + n \log n + m)}$
Ada-IPFE	$\mathbf{O(n^{\log_2 7})}$

Secondly, as illustrated in Fig. 8, the execution time of the HE scheme increases significantly with the vector dimension, which matches the theoretical analysis in Table VI. This rise is due to the need for n homomorphic multiplications and $n-1$

TABLE VII
COMPARISON OF COMMUNICATION OVERHEAD IN QKV VECTOR CALCULATION ACROSS DIFFERENT MODELS

Model	CipheRAG	OT	HE
GPT2	1.1 MB	15.5 MB	12.7 MB
Qwen2	7 MB	404.2 MB	25.28 MB
Llama3	16 MB	901.45 MB	50.61 MB

homomorphic additions during the calculation. In contrast, the Ada-IPFE method demonstrates remarkably stable and low execution times, remaining unaffected by increasing vector dimensions, which highlights its high efficiency. While OT also maintains relatively fast execution times with only a slight upward trend, its efficiency is primarily limited by communication overhead. As shown in Table VII, the communication overhead for QKV computation across different models is significantly lower when using the Ada-IPFE approach. This efficiency stems from Ada-IPFE's streamlined design, requiring only one-way encrypted uploads from the knowledge provider, eliminating the need for complex interactions. In contrast, the HE-based method requires the transmission of significantly larger ciphertexts due to the full HE operations, while OT-based methods involve multiple rounds of interactive communication between parties. These additional communication requirements result in considerably higher overhead for both HE and OT when compared to Ada-IPFE. While Ada-IPFE offers significantly lower communication overhead compared to HE and OT schemes, its security model assumes trustworthy key distribution and protected query issuance. For most real-world applications, including healthcare, finance, and enterprise deployments, this assumption is practically reasonable and aligns with moderate trust environments. However, in scenarios with highly adversarial threat models, such as military-grade or zero-trust infrastructures, stronger cryptographic guarantees (e.g., full semantic security) may be required. In such cases, FHE-based protocols remain viable alternatives, albeit with higher computational costs.

In Fig. 7, we simultaneously stress-test three axes of noise, *corpus contamination* (the fraction of unrelated documents injected into the index: 0% , 20% , and 50%), *query drop* (the proportion of true query tokens randomly omitted: 0% , 20% , and 40%), and *query add* (the fraction of spurious tokens inserted into the query: 0% and 20%). Across all contamination levels, the curves for $add = 0\%$ and $add = 20\%$ nearly coincide, indicating that inserting one extra token for every five genuine ones has virtually no effect on *hit@1*. Even under the extreme condition of dropping 40% of query tokens, an unusually severe perturbation, recall declines by only about 12% , a degradation that remains well within acceptable bounds. Finally, raising the contamination ratio from 0% to 50% (i.e. adding one irrelevant document for every two relevant ones) causes only a marginal performance drop. Together, these results demonstrate that our CipheRAG is remarkably robust: it tolerates heavy token deletion, moderate token insertion, and substantial corpus "chaff" without catastrophic loss of retrieval accuracy.

In conclusion, CipheRAG adopts Ada-IPFE for encryption due to its efficiency, low communication overhead, and scalability. Unlike HE and OT, which require complex interactions and incur significant costs, Ada-IPFE enables one-way encrypted uploads, simplifying deployment while maintaining robust privacy protection. Its low computational complexity and stable performance across vector dimensions make it ideal for real-time and large-scale applications.

VIII. DISCUSSION

Our proposed framework not only enhances the security of RAG systems but also demonstrates strong practical utility across privacy-sensitive domains such as healthcare and finance. CipheRAG enables efficient and privacy-preserving access to encrypted knowledge, supporting regulatory compliance in settings such as HIPAA-constrained medical environments. Experiments confirm that our design achieves up to $35\times$ speedups in token generation and significantly lower communication overhead compared to FHE or OT-based schemes. Moreover, CipheRAG's blockchain-based design promotes decentralized and tamper-evident knowledge sharing, ensuring transparency and auditability in collaborative scenarios. This property is particularly valuable in multi-party workflows such as federated healthcare or cross-institutional research. By balancing privacy, efficiency, and usability, CipheRAG offers a viable direction for deploying secure AI systems in real-world settings. Its empirical performance and architectural flexibility suggest promising avenues for standardization and policy development in privacy-preserving generative AI.

IX. CONCLUSION

In this paper, we presented CipheRAG, an efficient vector-multiplicative privacy-preserving RAG framework for large language models. Our framework tackles key challenges in secure knowledge retrieval and generation by combining ALSH with Ada-IPFE, enabling both privacy and performance. The proposed decryption-enabled attention mechanism further allows encrypted knowledge to be seamlessly incorporated into the language model's generation process without compromising efficiency. Comprehensive experiments confirm that CipheRAG achieves substantial gains in computational speed while preserving strong privacy guarantees, offering a practical balance suitable for real-world applications.

Despite these contributions, we acknowledge certain limitations regarding the threat model. While CipheRAG minimizes unintended data exposure by ensuring that only necessary decryption results are revealed, potential risks remain in scenarios where the LLM provider is not fully trusted. Specifically, adversaries could potentially infer original input information from intermediate states such as QKV vectors or the decrypted outputs of linear layers. To address this, future work may integrate existing privacy-enhancing techniques, such as adversarial training or differential privacy [60], [61], to process these initial layers. These measures would ensure that intermediate outputs cannot be exploited to recover sensitive data, thereby further reinforcing the system against privacy leakage.

REFERENCES

[1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 9459–9474.

[2] T. B. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.

[3] Y. Wang, Y. Pan, M. Yan, Z. Su, and T. H. Luan, "A survey on ChatGPT: AI-generated contents, challenges, and solutions," *IEEE Open J. Comput. Soc.*, vol. 4, pp. 280–302, 2023.

[4] Y. Wang et al., "Large model agents: State-of-the-art, cooperation paradigms, security and privacy, and future trends," *IEEE Commun. Surv. Tuts.*, vol. 28, pp. 1906–1949, 2026.

[5] E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education," *Learn. Individual Differences*, vol. 103, 2023, Art. no. 102274.

[6] W. Fan et al., "A survey on RAG meeting LLMs: Towards retrieval-augmented large language models," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2024, pp. 6491–6501.

[7] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "Retrieval augmented language model pre-training," in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, pp. 3929–3938.

[8] Y.-H. Huang, Y. Tsai, H. Hsiao, H.-Y. Lin, and S.-D. Lin, "Transferable embedding inversion attack: Uncovering privacy risks in text embeddings without model queries," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 4193–4205.

[9] W. Zou, R. Geng, B. Wang, and J. Jia, " {PoisonedRAG }: Knowledge corruption attacks to { Retrieval-Augmented} generation of large language models," in *Proc. 34th USENIX Secur. Symp.*, 2025, pp. 3827–3844.

[10] G. Feretzakis, K. Papaspyridis, A. Gkoulalas-Divanis, and V. S. Verykios, "Privacy-preserving techniques in generative AI and large language models: A narrative review," *Information*, vol. 15, no. 11, 2024, Art. no. 697.

[11] G. Feretzakis et al., "Securing a generative AI-powered healthcare chatbot," *Stud. Health Technol. Informat.*, vol. 321, pp. 195–199, 2024.

[12] G. Feretzakis and V. S. Verykios, "Trustworthy AI: Securing sensitive data in large language models," *AI*, vol. 5, no. 4, pp. 2773–2800, 2024.

[13] C. Dwork, "Differential privacy," in *Proc. Int. Colloq. Automata, Lang., Program.*, 2006, pp. 1–12.

[14] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. 23rd Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2017, pp. 409–437.

[15] B. Knott, S. Venkataraman, A. Hannun, S. Sengupta, M. Ibrahim, and L. van der Maaten, "CrypTen: Secure multi-party computation meets machine learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 4961–4973.

[16] X. Hou et al., "CipherGPT: Secure two-party GPT inference," *IEEE Trans. Dependable Secure Comput.*, pp. 1–16, 2026.

[17] L. de Castro, D. Escudero, A. Agrawal, A. Polychroniadou, and M. Veloso, "EncryptedLLM: Privacy-preserving large language model inference via GPU-accelerated fully homomorphic encryption," in *Proc. 42nd Int. Conf. Mach. Learn.*, 2025, pp. 12677–12688.

[18] Z. Fu et al., "EncChain: Enhancing large language model applications with advanced privacy preservation techniques," *Proc. VLDB Endowment*, vol. 17, no. 12, pp. 4413–4416, 2024.

[19] D. Zhao, "FRAG: Toward federated vector database management for collaborative and secure retrieval-augmented generation," 2024, arXiv:2410.13272.

[20] W. Lu et al., "BumbleBee: Secure two-party inference framework for large transformers," in *Proc. 32nd Annu. Netw. Distrib. Syst. Secur. Symp.*, 2025, pp. 1–15.

[21] B. Wang, Q. Lou, M. Zheng, and D. Zhao, "PIR-RAG: A system for private information retrieval in retrieval-augmented generation," 2025, arXiv:2509.21325.

[22] F. Chen, P. Li, S. Pan, L. Zhong, and J. Deng, "Giant could be tiny: Efficient inference of giant models on resource-constrained UAVs," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21170–21179, Jun. 2024.

[23] Y. Qin et al., "MECLA: Memory-compute-efficient LLM accelerator with scaling sub-matrix partition," in *Proc. ACM/IEEE 51st Annu. Int. Symp. Comput. Architecture*, 2024, pp. 1032–1047.

[24] A. Shrivastava and P. Li, "Asymmetric LSH (ALSH) for sublinear time maximum inner product search (MIPS)," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 2321–2329.

[25] H. Yang, Y. Su, J. Qin, and H. Wang, "Privacy-preserving outsourced inner product computation on encrypted database," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 1320–1337, Mar./Apr. 2022.

[26] A. Vaswani, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.

[27] S. Borgeaud et al., "Improving language models by retrieving from trillions of tokens," in Proc. Int. Conf. Mach. Learn., 2022, pp. 2206–2240.

[28] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *Proc. 16th Conf. Eur. Chapter Assoc. Comput. Linguistics*, 2021, pp. 874–880.

[29] Y. Yang, L. Wu, and G. Li, "A survey on security and privacy issues in Internet-of-Things," *IEEE Internet Things J.*, vol. 4, no. 5, pp. 1250–1258, Oct. 2017.

[30] N. Carlini et al., "Extracting training data from large language models," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur. 21)*, 2021, pp. 2633–2650.

[31] M. Nasr et al., "Scalable extraction of training data from (production) language models," in *Proc. Int. Conf. Learn. Represent.*, 2025, pp. 1–17.

[32] M. Duan et al., "Do membership inference attacks work on large language models?" 2024, arXiv:2402.07841.

[33] B. C. Das, M. H. Amini, and Y. Wu, "Security and privacy challenges of large language models: A survey," *ACM Comput. Surv.*, vol. 57, no. 6, pp. 1–39, 2025.

[34] J. Morris, V. Kuleshov, V. Shmatikov, and A. M. Rush, "Text embeddings reveal (almost) as much as text," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 12448–12460.

[35] H. Li, J. Wu, H. Xu, G. Li, and M. Guizani, "Explainable intelligence-driven defense mechanism against advanced persistent threats: A joint edge game and AI approach," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 757–775, Mar./Apr. 2022.

[36] G. Ren, J. Wu, G. Li, S. Li, and M. Guizani, "Protecting intellectual property with reliable availability of learning models in AI-based cybersecurity services," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 2, pp. 600–617, Mar./Apr. 2024.

[37] J. Zhu, L. Patel, M. Zaharia, and R. A. Popa, "Compass: Encrypted semantic search with high accuracy," in *Proc. USENIX Symp. Oper. Syst. Des. Implement.*, 2025, pp. 915–938.

[38] G. Zyskind, T. South, and A. Pentland, "Don't forget private retrieval: Distributed private similarity search for large language models," in *Proc. Workshop Privacy Natural Lang. Process.*, 2024, pp. 7–19.

[39] Z. Ruoyan, Z. Zhongxiang, and B. Wankang, "Practical secure inference algorithm for fine-tuned large language model based on fully homomorphic encryption," 2025, arXiv:2501.01672.

[40] Y. Bae et al., "Privacy-preserving LLM interaction with socratic chain-of-thought reasoning and homomorphically encrypted vector databases," 2025, arXiv:2506.17336.

[41] X. Lin, J. Wu, J. Li, C. Sang, S. Hu, and M. J. Deen, "Heterogeneous differential-private federated learning: Trading privacy for utility truthfully," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 5113–5129, Nov./Dec. 2023.

[42] J. Yu, J. Zhou, Y. Ding, L. Zhang, Y. Guo, and H. Sato, "Textual differential privacy for context-aware reasoning with large language model," in *Proc. IEEE 48th Annu. Comput., Softw., Appl. Conf.*, 2024, pp. 988–997.

[43] T. Koga, R. Wu, Z. Zhang, and K. Chaudhuri, "Privacy-preserving retrieval-augmented generation with differential privacy," 2024, arXiv:2412.04697.

[44] S. Zeng et al., "Mitigating the privacy issues in retrieval-augmented generation (RAG) via pure synthetic data," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2025, pp. 24538–24569.

[45] J. Majmudar, C. Dupuy, C. Peris, S. Smaili, R. Gupta, and R. Zemel, "Differentially private decoding in large language models," 2022, arXiv:2205.13621.

[46] M. Datar, N. Immorlica, P. Indyk, and V. S. Mirrokni, "Locality-sensitive hashing scheme based on P-stable distributions," in *Proc. 20th Annu. Symp. Comput. Geometry*, 2004, pp. 253–262.

[47] P. Mohassel and Y. Zhang, "SecureML: A system for scalable privacy-preserving machine learning," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 19–38.

[48] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[49] Y. Psaras and D. Dias, "The interplanetary file system and the filecoin network," in *Proc. 50th Annu. IEEE-IFIP Int. Conf. Dependable Syst. Netw.-Supplemental Vol.*, 2020, pp. 80–80.

[50] J. D. M.-W. C. Kenton and L. K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Assoc. Comput. Linguistics, Hum. Lang. Technol.*, 2019, pp. 4171–4186.

[51] N. Reimers, "Sentence-Bert: Sentence embeddings using siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. Int. Joint Conf. Natural Lang. Process.*, 2019, pp. 3982–3992.
[52] Q. Huang, G. Ma, J. Feng, Q. Fang, and A. K. Tung, "Accurate and fast asymmetric locality-sensitive hashing scheme for maximum inner product search," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 1561–1570.
[53] T. Kwiatkowski et al., "Natural questions: A benchmark for question answering research," *Trans. Assoc. Comput. Linguistics*, vol. 7, pp. 453–466, 2019.
[54] A. Dubey et al., "The Llama 3 herd of models," 2024, arXiv:2407.21783.
[55] J. Bai et al., "Qwen technical report," 2023, arXiv:2309.16609.
[56] V. Cohen and A. Gokaslan, "OpenGPT-2: Open language models and implications of generated text," *XRDS, Crossroads, ACM Mag. Students*, vol. 27, no. 1, pp. 26–30, 2020.
[57] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," in *Proc. Findings Assoc. Comput. Linguistics*, 2024, pp. 2318–2335.
[58] A. Patra, T. Schneider, A. Suresh, and H. Yalame, "ABY2. 0: Improved mixed-protocol secure two-party computation," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2165–2182.
[59] E. B. Barker, W. C. Barker, W. E. Burr, W. T. Polk, and M. E. Smid, "SP 800-57. Recommendation for key management, part 1: General (revised)," NIST Special Publication 800-57, National Institute of Standards and Technology, 2007.
[60] T. Ryffel, E. Dufour-Sans, R. Gay, F. Bach, and D. Pointcheval, "Partially encrypted machine learning using functional encryption," 2019, arXiv:1905.10214.
[61] J. Zhou, Z. Su, Y. Wang, and J. Wu, "DM-DPL: Toward discrete matrixing differentially private learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 6149–6161, 2025.

Jinhao Zhou (Graduate Student Member, IEEE) received the MS degree from the School of Cyber Science and Engineering, Xi'an Jiaotong University, Xi'an, China. He is currently working toward the PhD degree with the Graduate School of Information, Production and System, Waseda University, Fukuoka, Japan. His research interests include privacy-preserving machine learning, federated learning, and large language models.

Jun Wu (Senior Member, IEEE) received the PhD degree in information and telecommunication studies from Waseda University, Japan, in 2011. He is currently a professor with the Graduate School of Information, Production and Systems, Waseda University. He is the chair of IEEE P21451-1-5 Standard Working Group for Internet of Things. He is the author or coauthor of more than 200 peer-reviewed journal/conference papers in his research fields, which include the intelligence and security techniques of Internet of Things (IoT), edge computing, Big Data, and 5 G/6 G. He has served as the general chair of IEEE SmartCloud 2024, PC chair of ISIPS 2023, track chair of VTC 2019, 2020, 2023 and TPC member of more than ten international conferences, such as ICC and GLOBECOM. He serves an associate editor for *IEEE Systems Journal* and *IEEE Networking Letters*.